



抖音启动性能监控平台建设实践

郭蕾

讲师自我介绍



郭蕾

2016年本科毕业，毕业后从事Android基础架构相关工作，2019年加入字节跳动，目前在抖音基础技术-质量团队负责抖音性能诊断平台的建设工作，专注于线上场景的问题挖掘与诊断。



01 平台建设背景

02 平台建设思路

03 阶段性成果

04 总结&展望

01

平台建设背景

01_启动性能监控平台背景



启动预期



监控背景



监控现状

01_启动预期



打开速度快

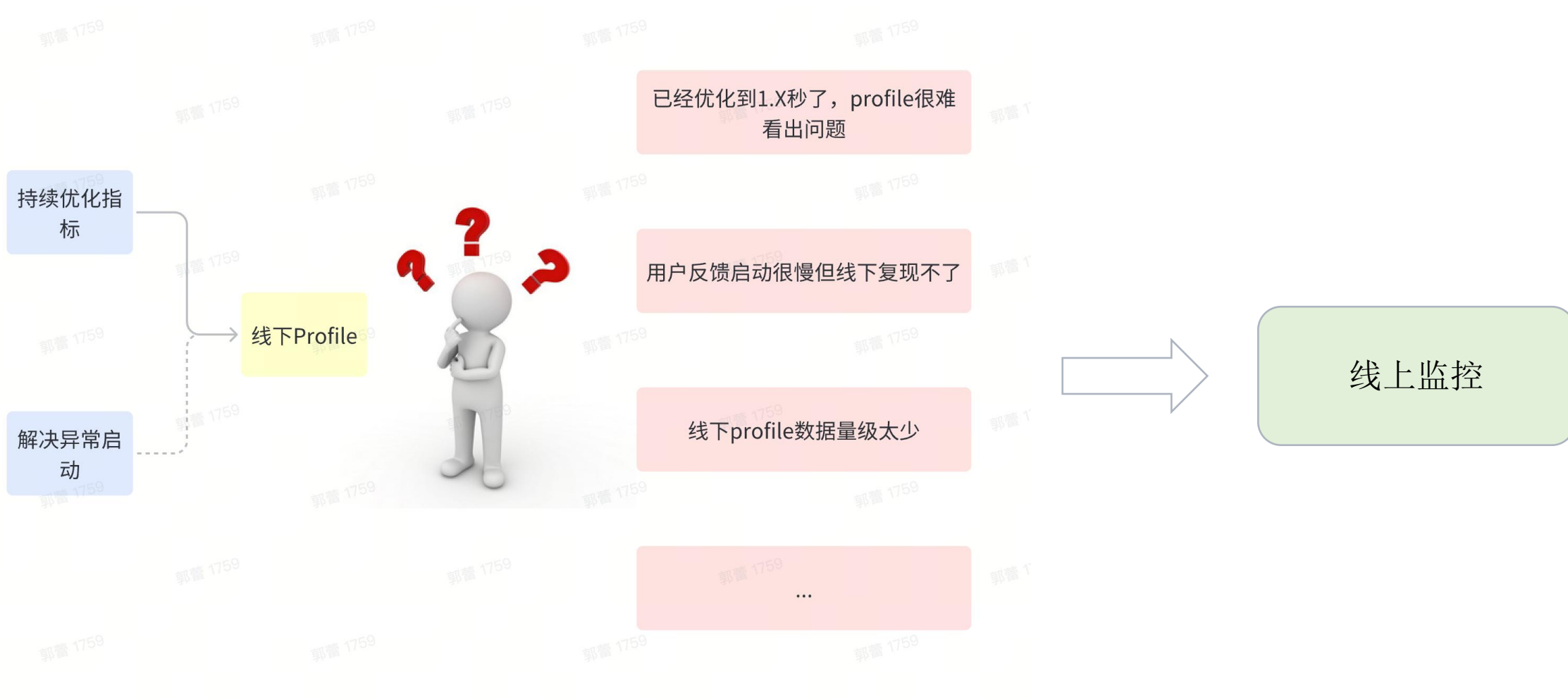
从不崩溃、
也不卡死

没有黑屏、
花屏

首刷内容赞

...

01_监控背景



01_监控现状

指标监控

手动埋点

使用埋点去建设线上指标看板，要增加很多埋点

异常监控

稳定性

关注稳定性，如启动ANR，但难以解决

性能监控

单点异常

关注异常性能数据，不够全面

01_ 监控现状-指标监控

通过埋点的方式监控启动阶段的耗时信息。

痛点：只有指标，不知道怎么优化。也无法感知用户遇到了哪些体验问题。

名称	样本量	平均值	pct25	pct50
app_start_to_main_focus	314.53 k	2.1 k	1.2 k	1.1 k
	14	1.2 k	1.2 k	1.1 k
	286.67 k	1.1 k	1.2 k	1.1 k
	12	1.2 k	1.2 k	1.1 k
	315.01 k	1.2 k	1.2 k	1.1 k
	12	1.2 k	1.2 k	1.1 k

01_ 监控现状-性能监控

单点的性能数据监控能力，如业内的Matrix慢函数等。

痛点：

- 堆栈信息不全、消费困难
- 缺少归因能力，不知道到底哪里耗时了



02

平台建设思路

02_平台建设思路



目标与价值



整体方案



技术建设

02_ 目标与价值

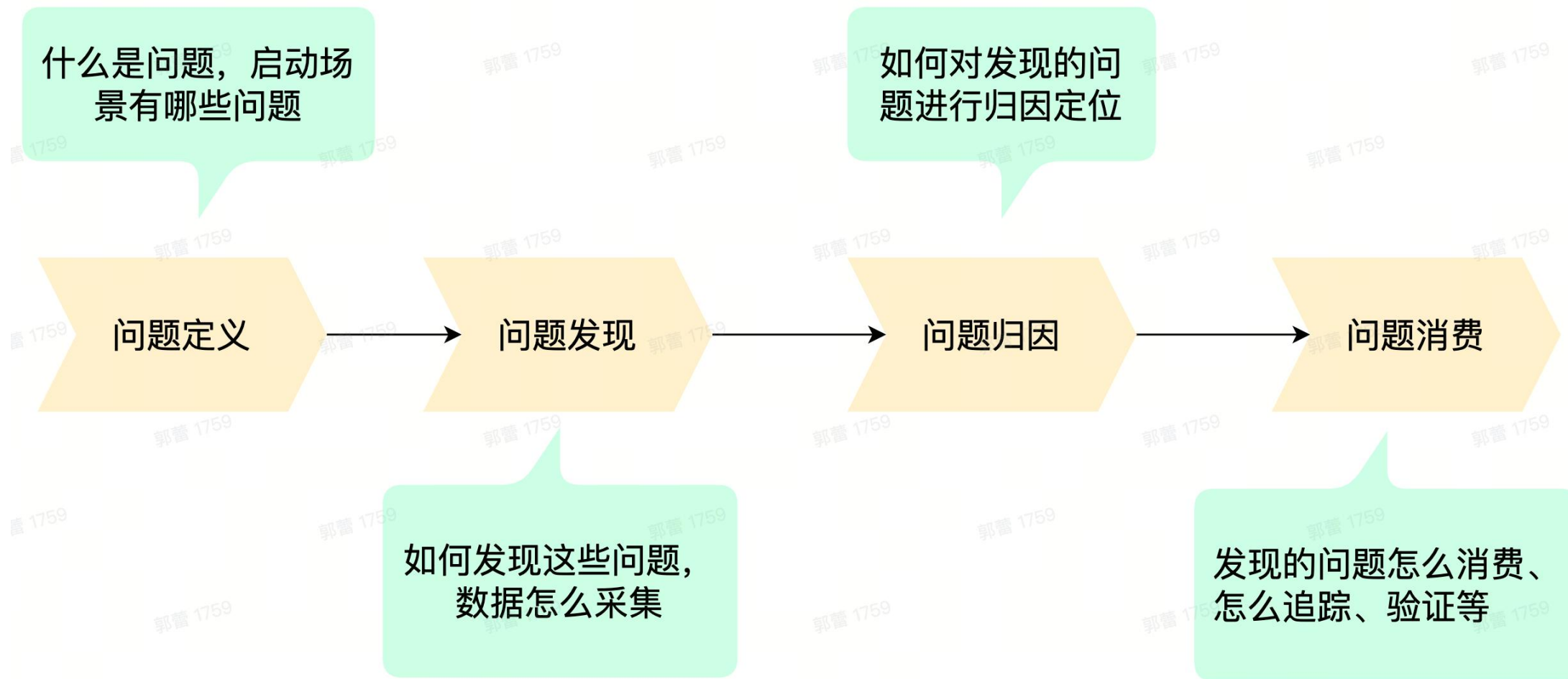
目标

1. 具备问题挖掘、归因和消费能力的**归因平台**
2. 发现启动相关的**边界问题**，如启动卡死

价值

挖掘存量问题，为优化提供**数据支撑**，提升用户体验，**创造业务价值**

02_整体方案设计-流程

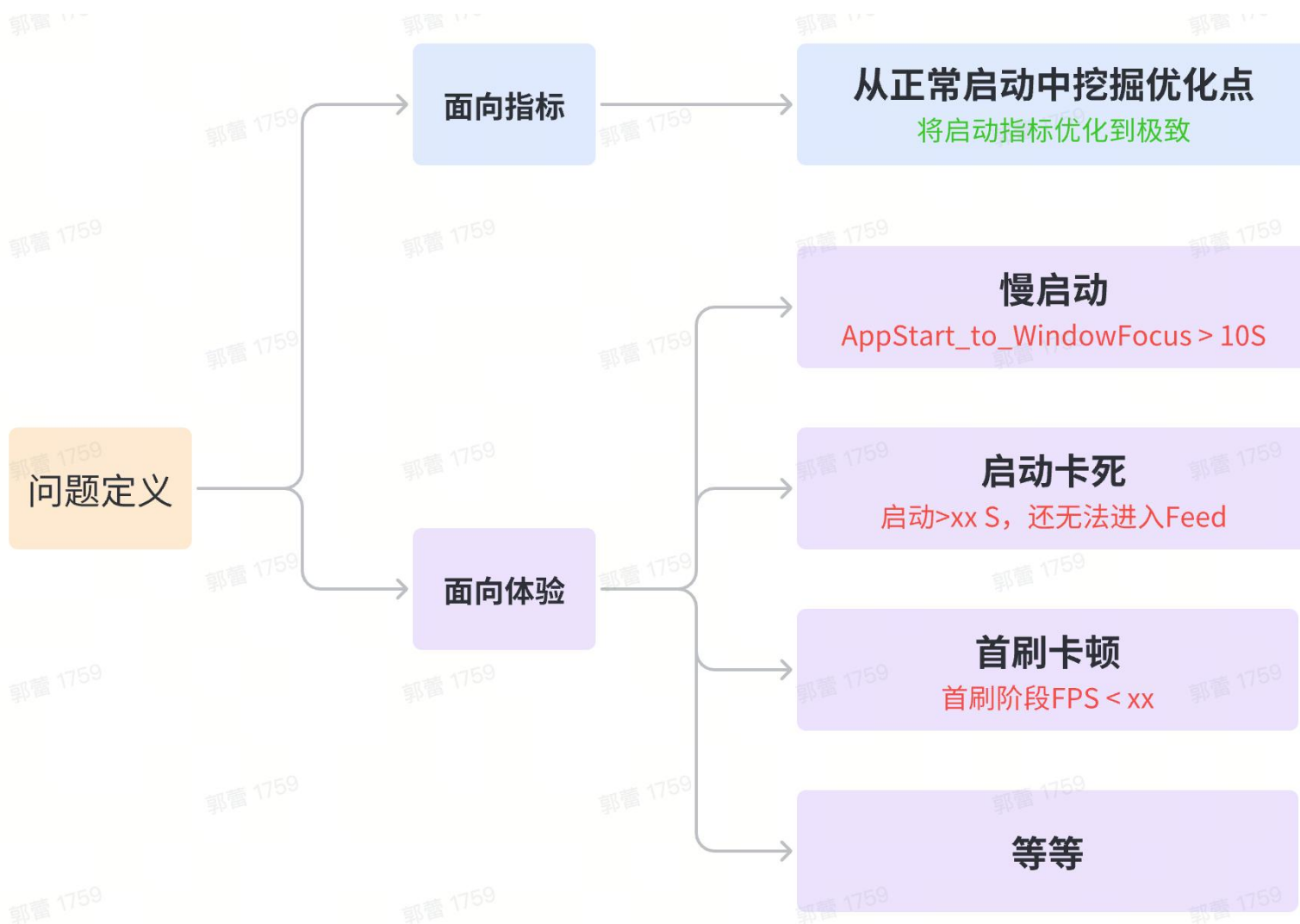


02_整体方案设计-架构

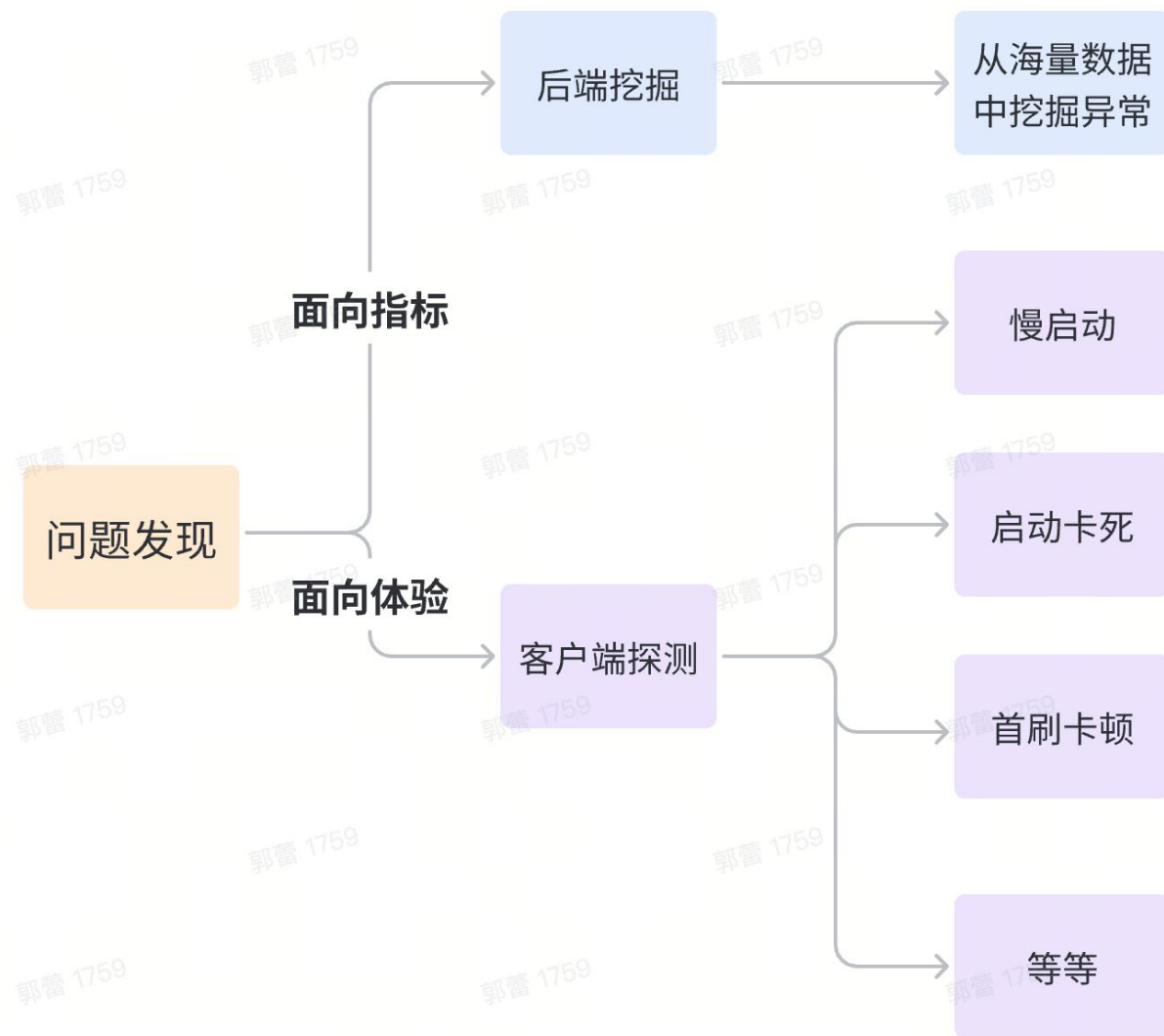


- 问题发现:
 - 基础能力: 底层依赖能力
 - 采集能力: 性能数据采集能力
 - 端策略: 采样、上报、发现策略等
- 问题归因: 数据聚合、异常挖掘、归因等
- 问题消费: 平台侧问题如何进行消费

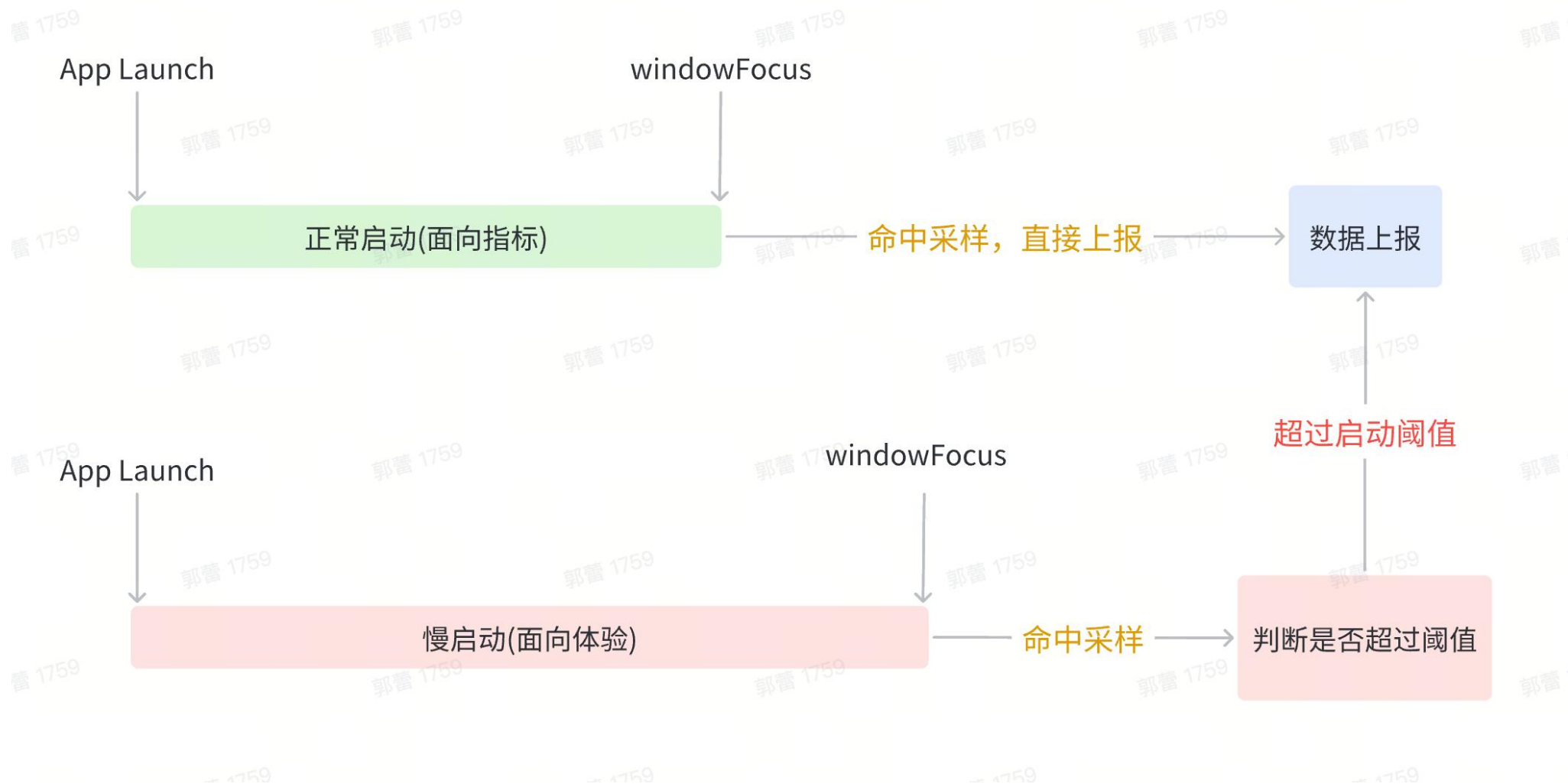
02_ 技术建设-问题定义



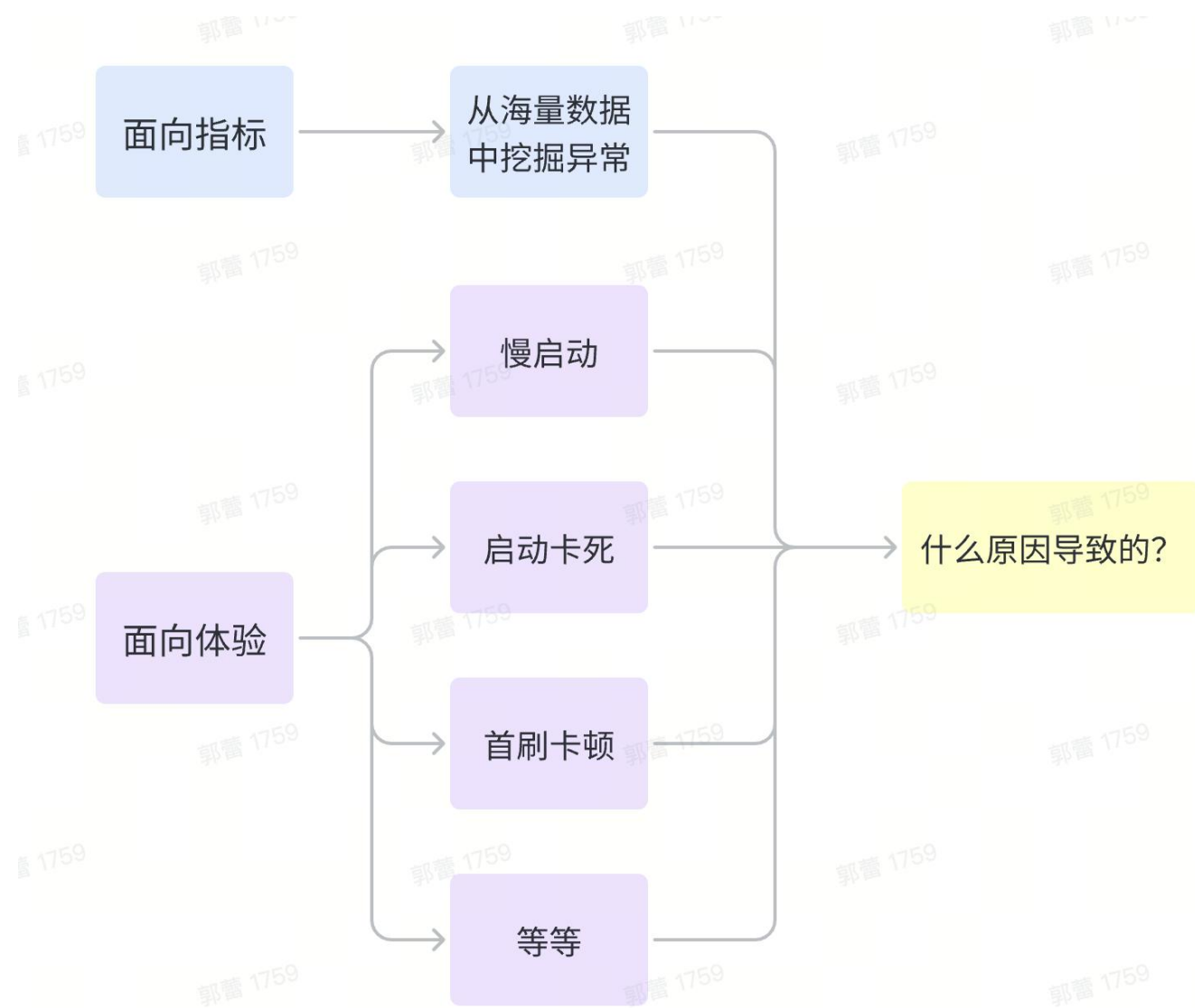
02_ 技术建设-问题发现



02_ 技术建设-问题发现



02_ 技术建设-问题归因



02_ 技术建设-问题归因



慢启动为例

主线程耗时

慢函数

CPU抢占

后台任务

内存分配、缺页
等

内存问题

其他

02_ 技术建设-问题归因-慢函数方案

慢函数使用插桩方案，即在编译时进行方法插桩，运行时进行记录时间信息。

全插桩

1. 对包体级影响较大
2. 对性能有比较大的影响，线下测试**19%**的性能损耗

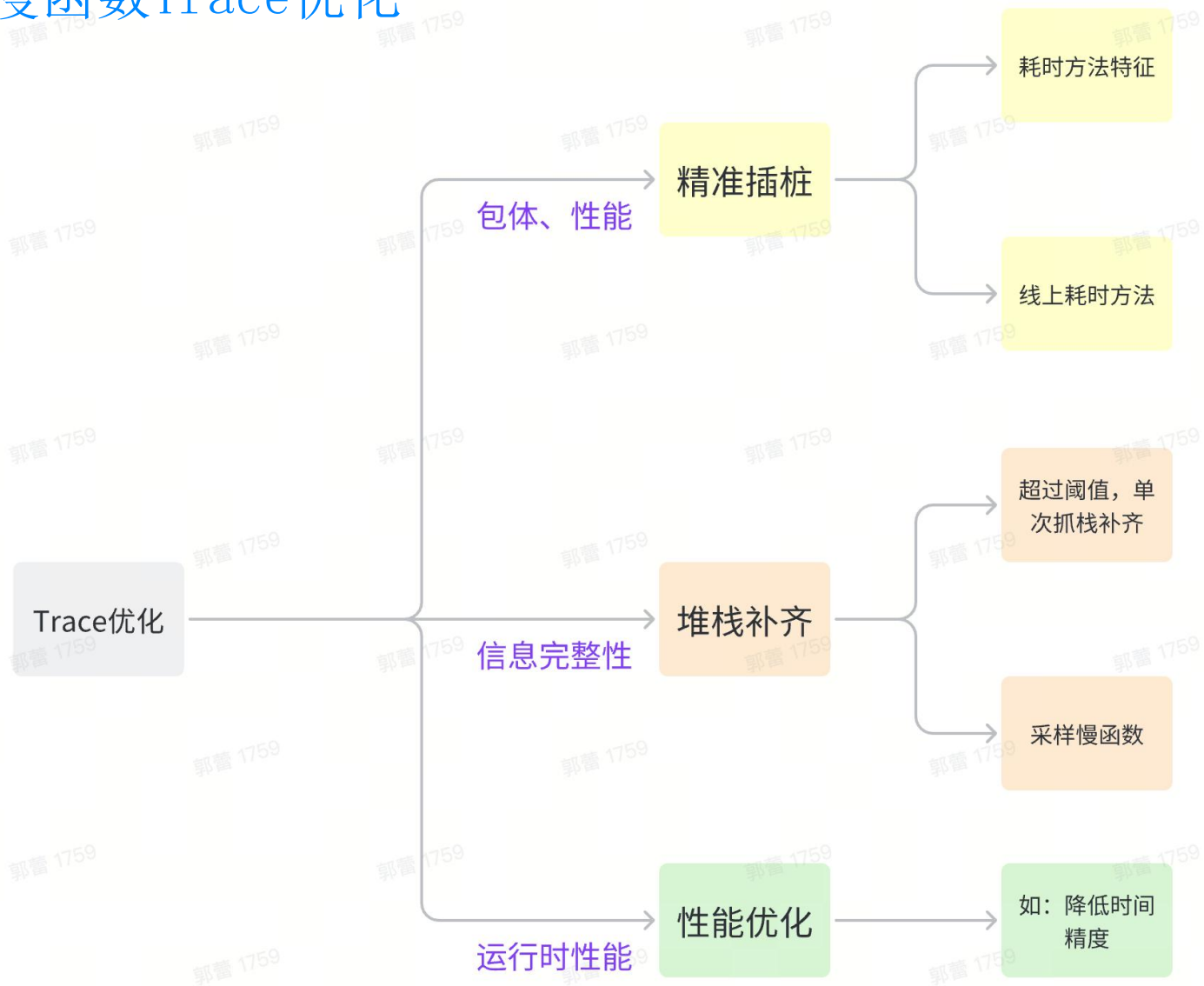
减少插桩(控制在100W插桩量)

1. 堆栈信息不全，缺失关键信息

插桩方案缺陷：

1. 无法对系统堆栈进行插桩，缺少系统堆栈信息

02_ 技术建设-问题归因-慢函数Trace优化



02_ 技术建设-问题归因-耗时方法特征

针对锁、循环、以及一些特殊方法调用的方法进行插桩。插桩量由100W下降到7W。

函数特征	指令名	说明
锁	monitorenter & monitorexit ; ACC_SYNCHRONIZED	Java 锁关键字
循环	goto	循环关键字，依据实际情况决定是否加入“疑似耗时操作”名单
特殊方法调用	InvokeVirtual; InvokeSpecial; InvokeStatic; InvokeInterface; InvokeDynamic;	存在 Native、IO、线程、渲染等相关等方法调用。
大方法	InvokeVirtual; InvokeSpecial; InvokeStatic; InvokeInterface; InvokeDynamic;	方法中调用方法数目超过 20 个。（或者 20 个以上）

02_ 技术建设-问题归因-堆栈补齐

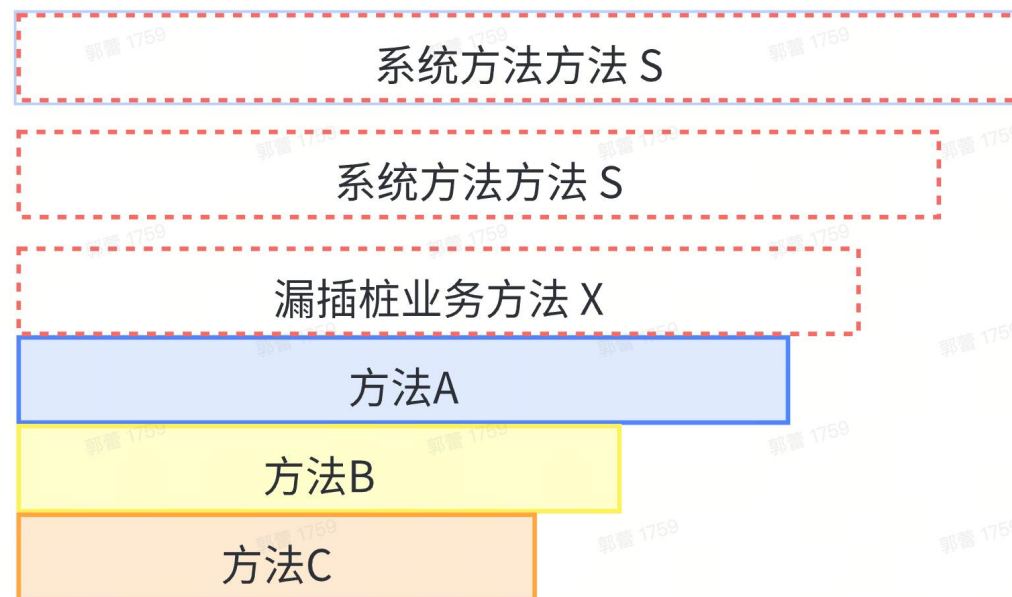
Dispatching.Message

缺失系统堆栈和业务堆栈

org.greenrobot.eventbus.EventBus.postSingleEventForEventType

插桩慢函数case

1. 我们减少了插桩量，必然会缺失信息
2. 系统堆栈无法插桩



02_ 技术建设-问题归因-堆栈补齐

在一些耗时或者具备耗时特征的调用(锁、IO等)结束时，进行一次抓栈。



02_ 技术建设-问题归因-精细化数据



```
bo ... ..getManager($Params)
```

android.content.Intent

▼ con

... .prashn.ii ushii aramshimanager@f...)

Section/Issue Long State

▼ com.xx.sdk.core.xx.getXXSync()

- ▼ `com.x.xxx.xxx.xx.xx.sdk.storage.dao.xx.getXX(java.lang.String,java.lang.String)`

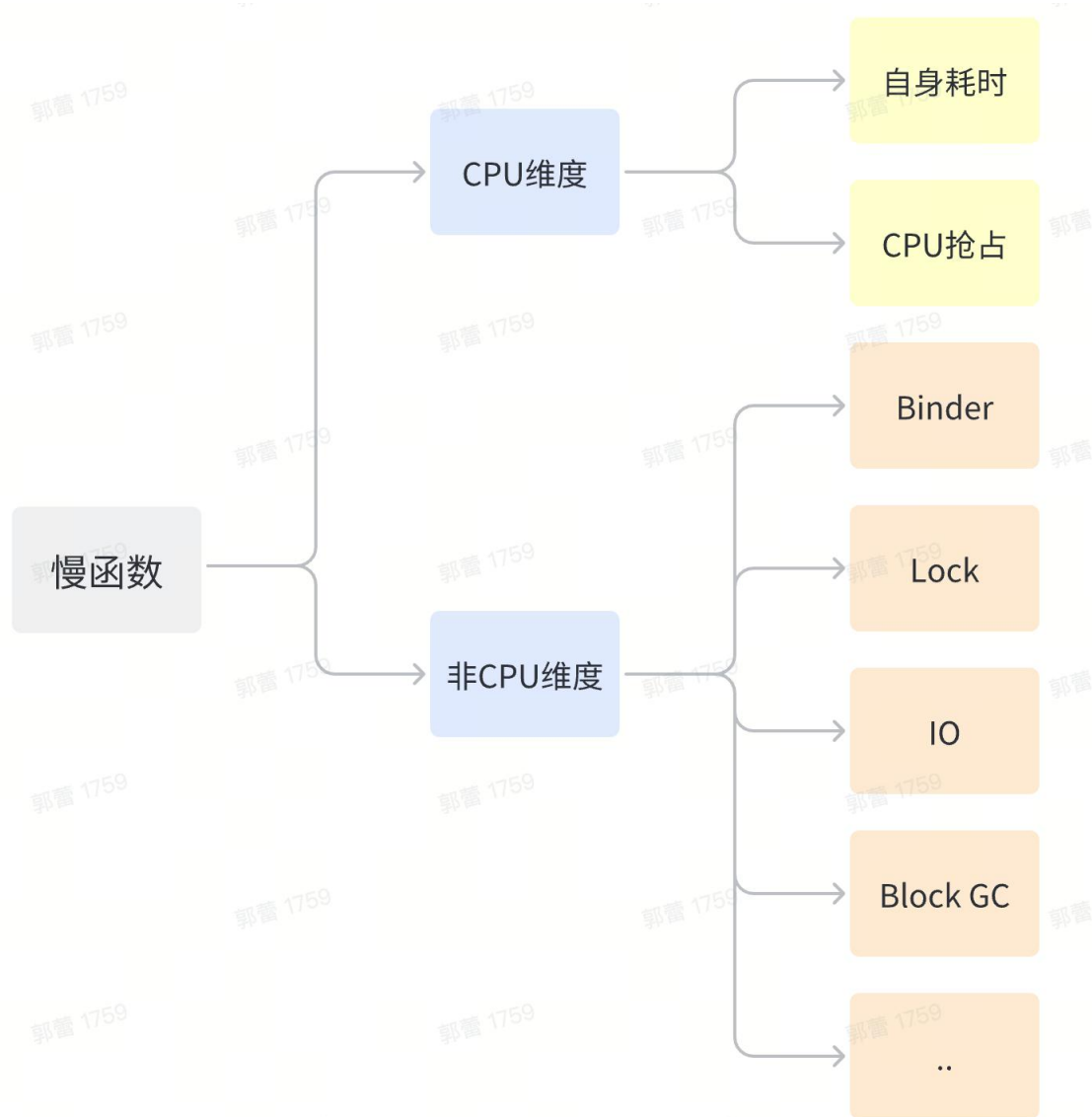
```
com.xx..DbManager.rawQuery(java.lang.String,java.lang.String[])
```

慢查询?

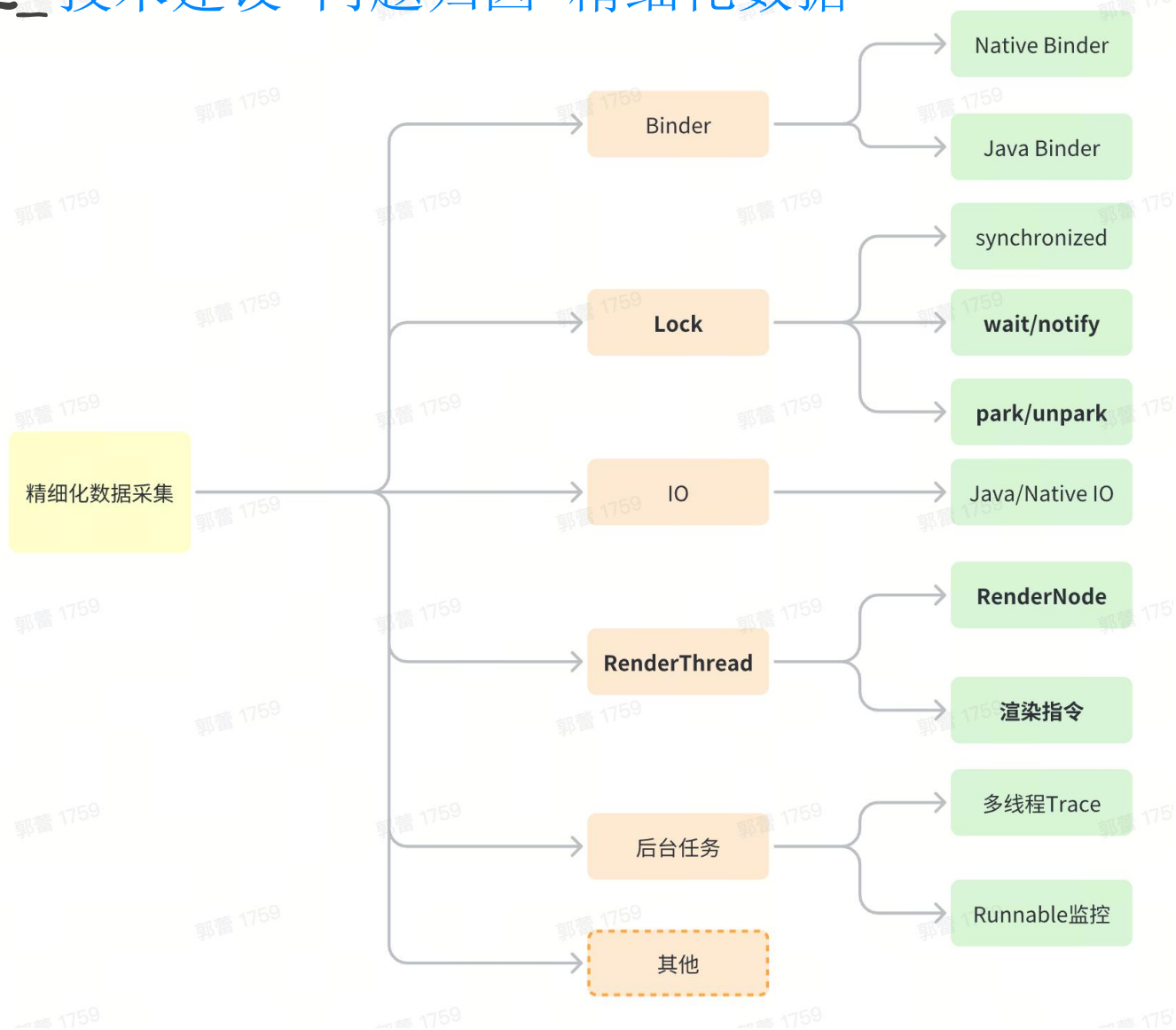
堆栈信息比较全，是否可以满足我们快速定位问题的诉求。

02_ 技术建设-问题归因-精细化数据

慢函数是怎么产生的。有哪些因素会导致慢函数。

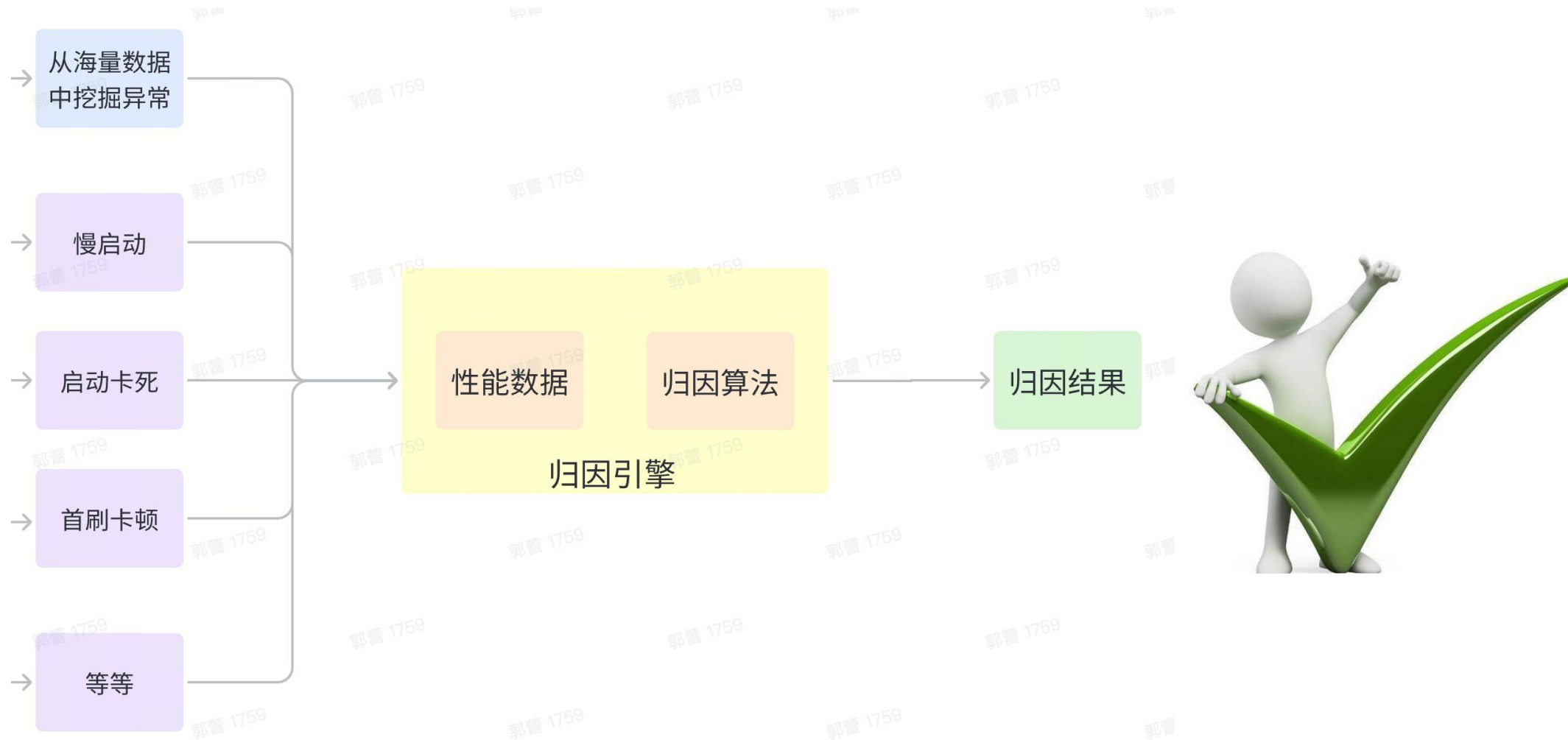


02_ 技术建设-问题归因-精细化数据

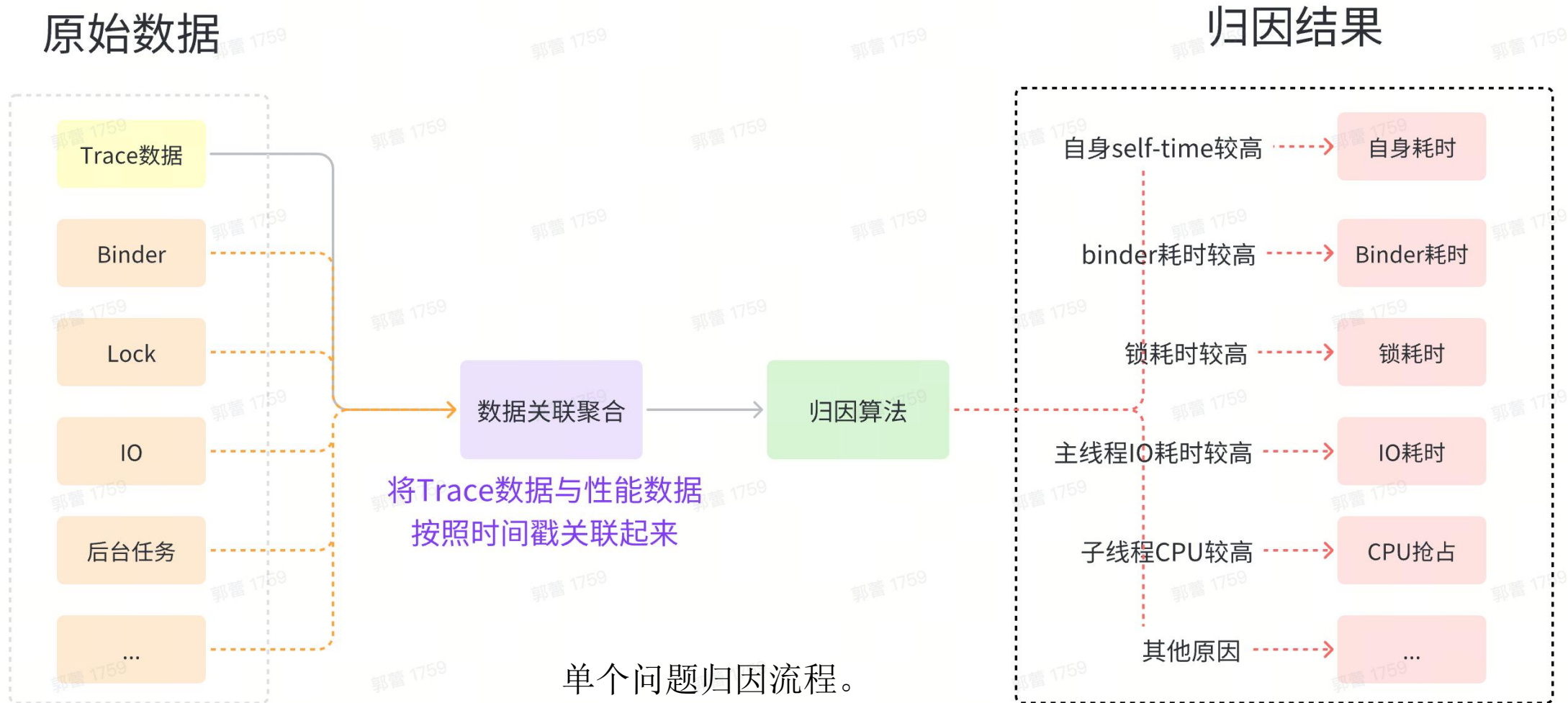


绝大部分能力会跟着btrace 2.0开源计划对外公布

02_ 技术建设-问题归因

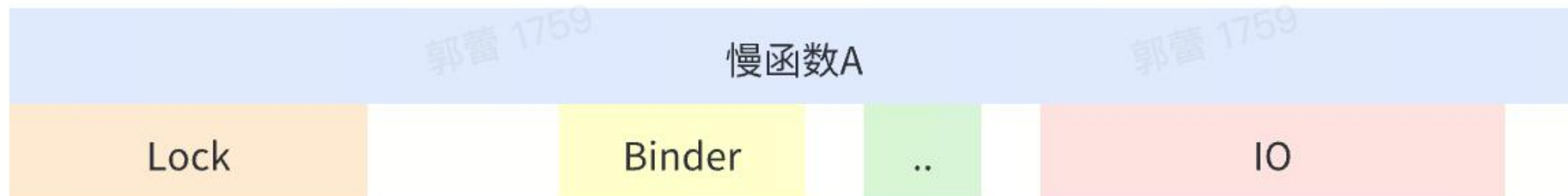


02_ 技术建设-问题归因-归因算法

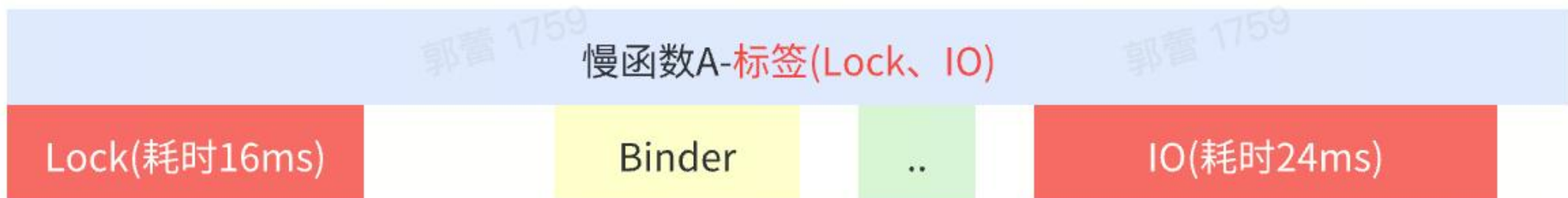


02_ 技术建设-问题归因-归因算法

1. 数据关联。



2. 归因打标，给单个case打上特征标签



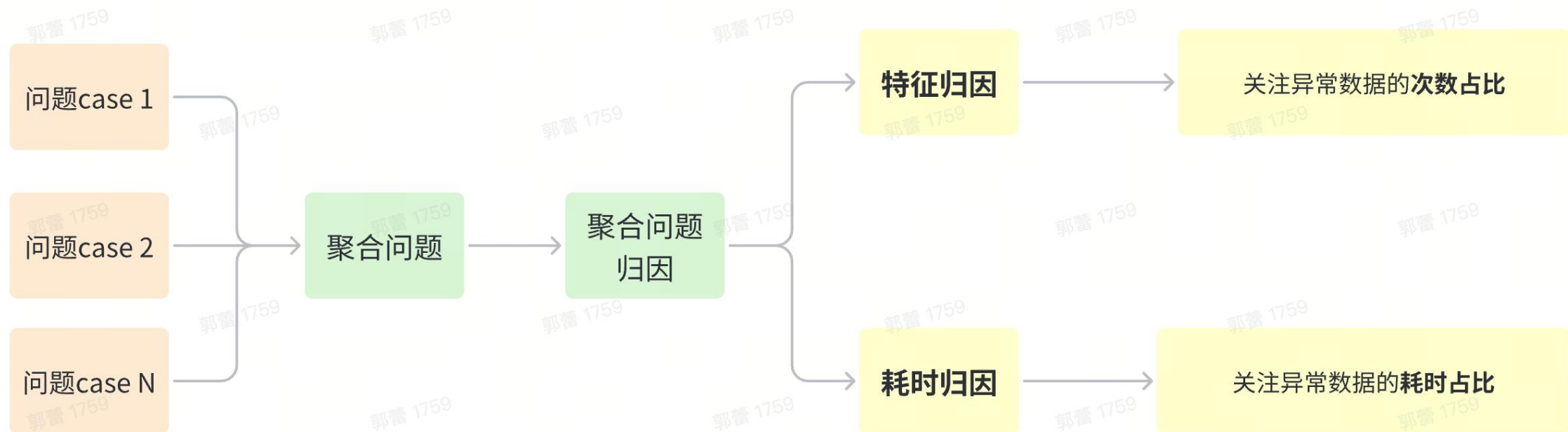
02_ 技术建设-问题归因-归因算法

归因说明 ?

标题	值	耗时	说明	评估
Lock 归因标签	关联的异常数据	78	超过阈值32ms	重度
		38	超过阈值32ms	重度
		14	超过阈值8ms	轻微
		9	超过阈值8ms	轻微
IO		194	超过阈值32ms	重度
		4005	子线程超过阈值2000ms	中度
		2961	子线程超过阈值2000ms	中度
		2369	子线程超过阈值2000ms	中度
		2139	子线程超过阈值2000ms	中度
Runnable				

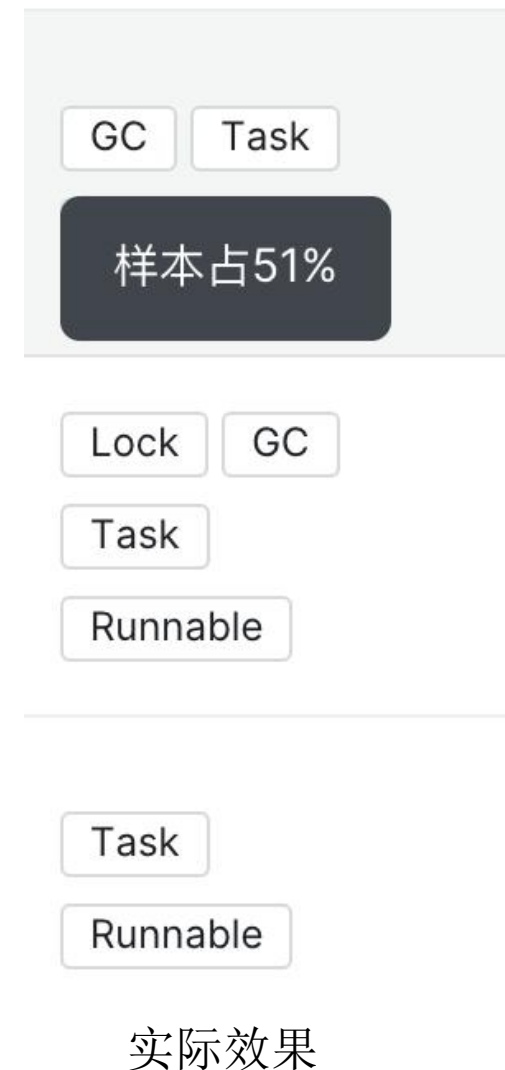
单个case归因结果

02_ 技术建设-问题归因-归因算法

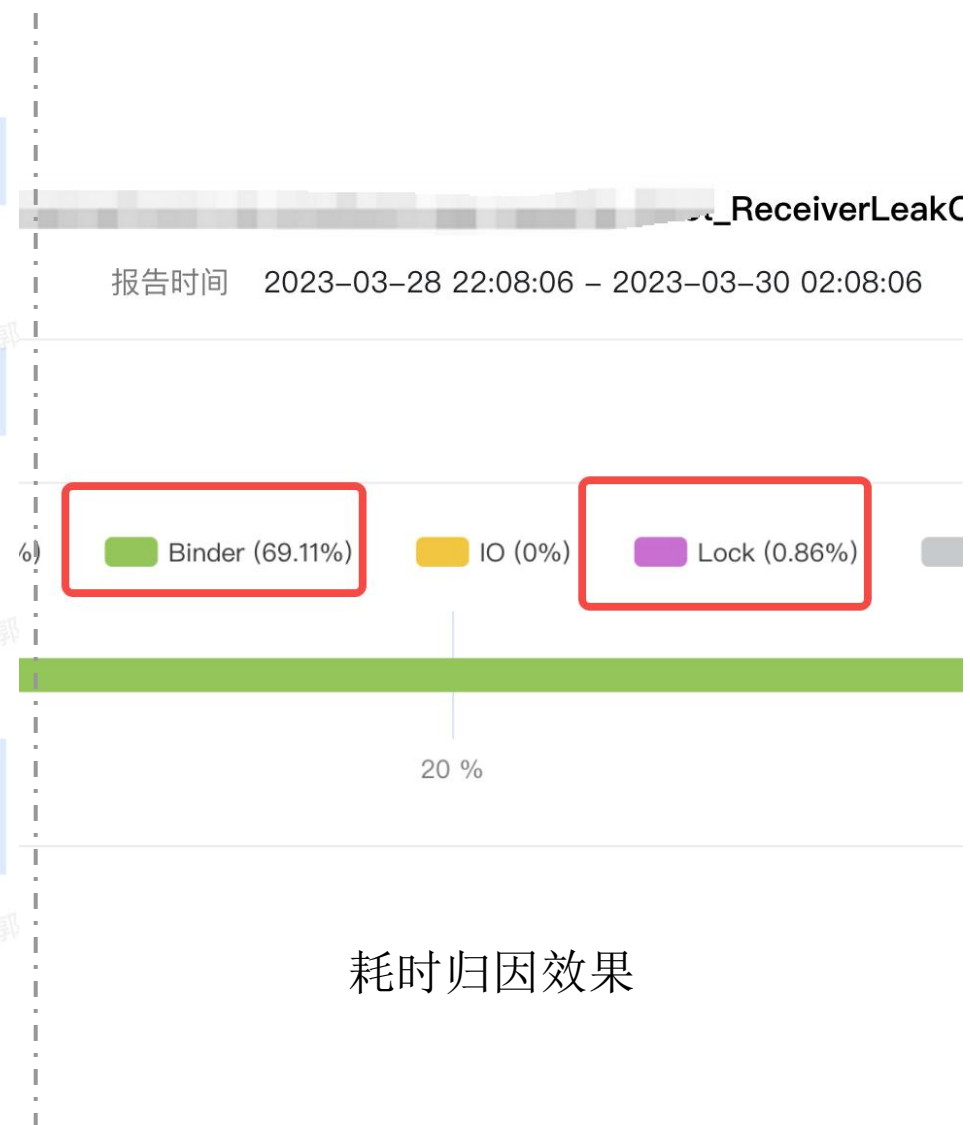


聚合问题的归因流程

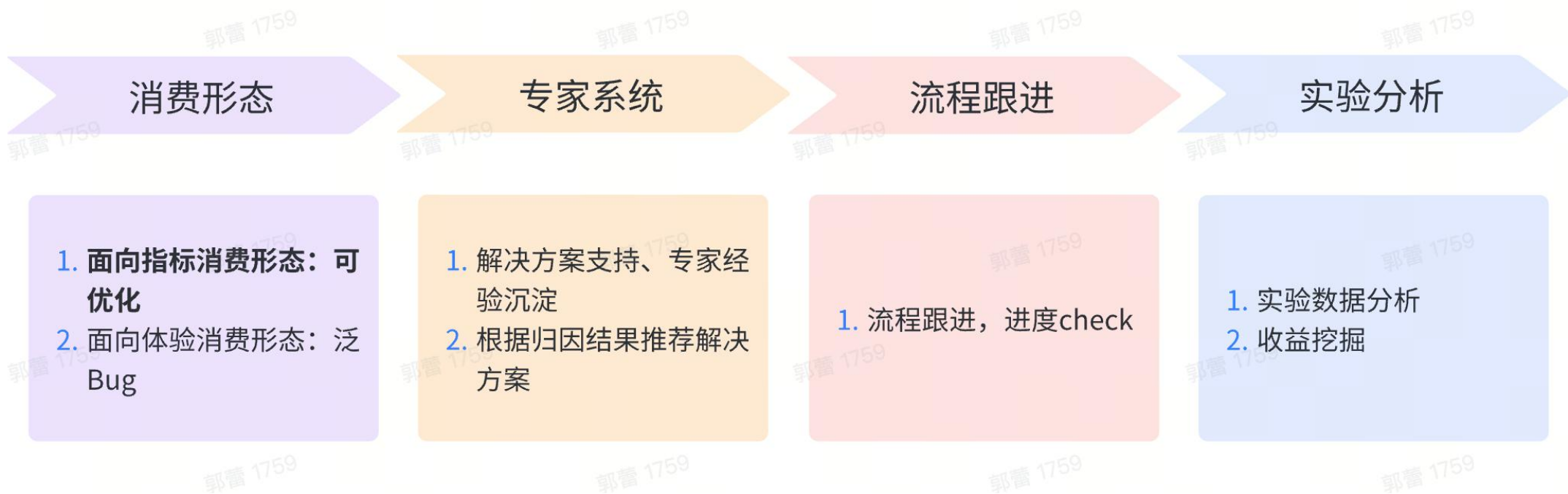
02_ 技术建设-问题归因-归因算法-特征归因



02_ 技术建设-问题归因-归因算法-耗时归因

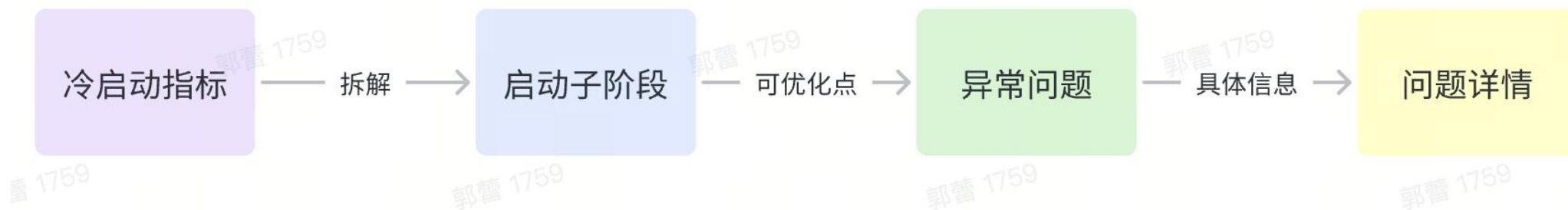


02_ 技术建设-问题消费

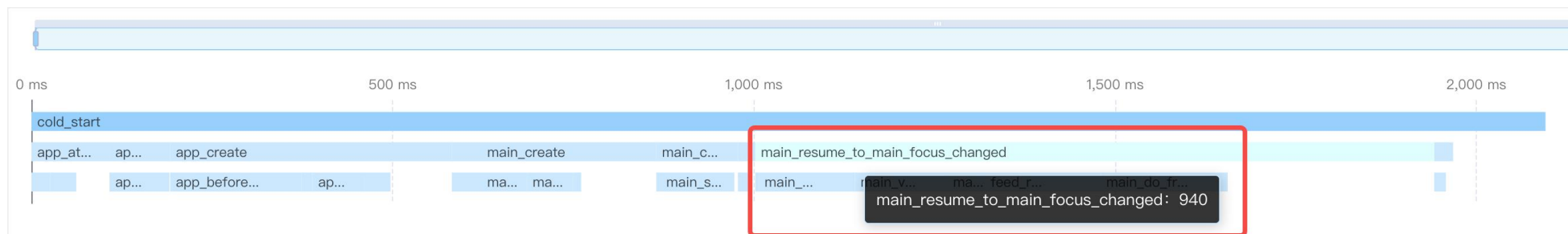


02_ 技术建设-问题消费-消费形态

面向指标的目标： 我们希望能从当前正常启动的用户中去挖掘更多的优化点。



启动阶段分布 点击阶段可查看对应的耗时分布



启动子阶段

02_ 技术建设-问题消费-消费形态

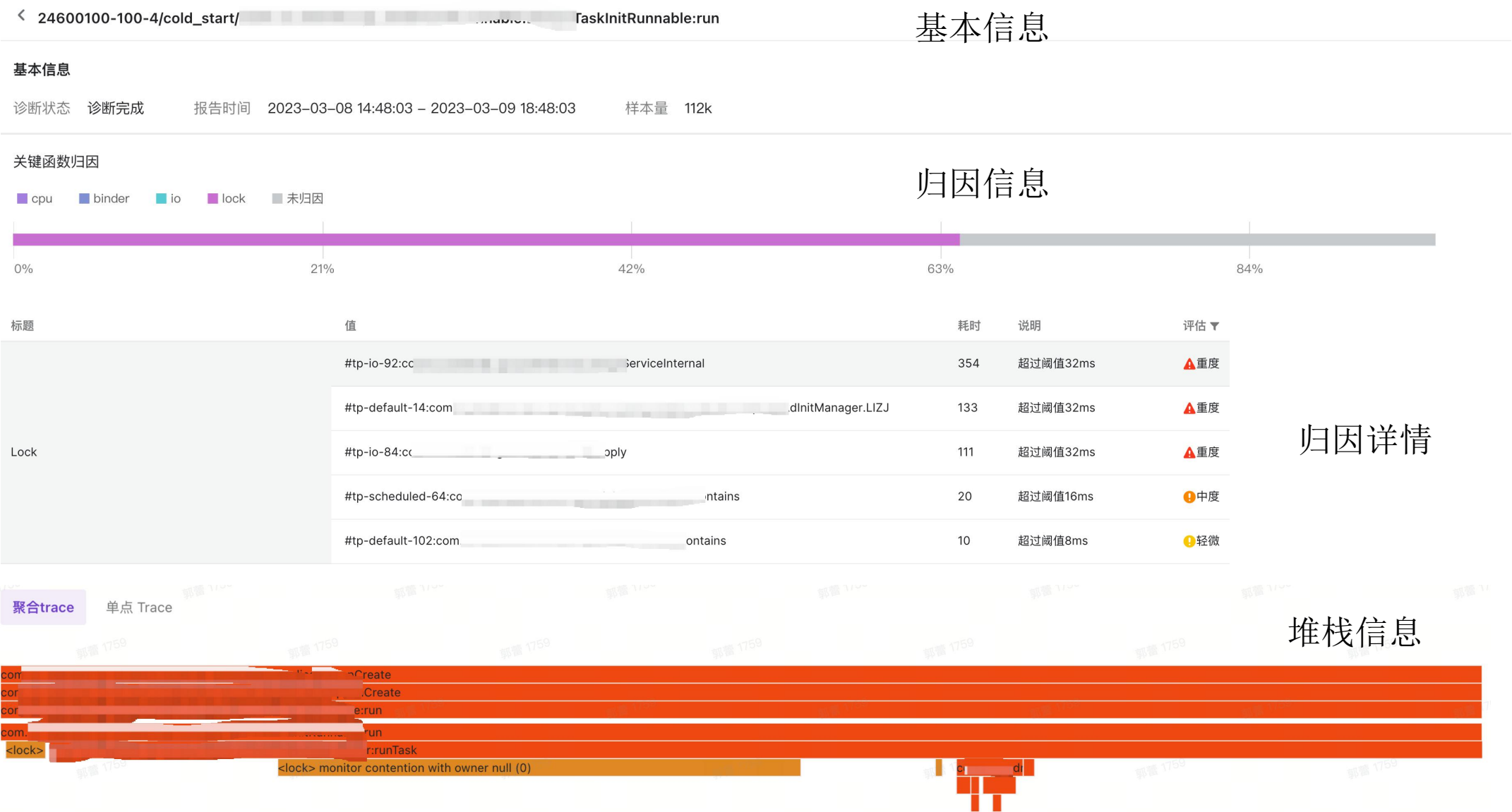
异常函数：函数self-time超过启动耗时1%的作为异常函数，这些函数是存在一定优化空间的。

函数名称	平均出现次数	耗时PCT50	累计影响用户耗时	耗时归因类型	耗时归因分布
me comme ance ate	1	620.5ms	48059ms	Lock	
ance ed	1	476ms	47987ms	未归因	
gerA d: anged	1	386ms	681329ms	未归因	
ko h	1	382.5ms	470528ms	未归因	
an eco s	1	378ms	295445ms	未归因	

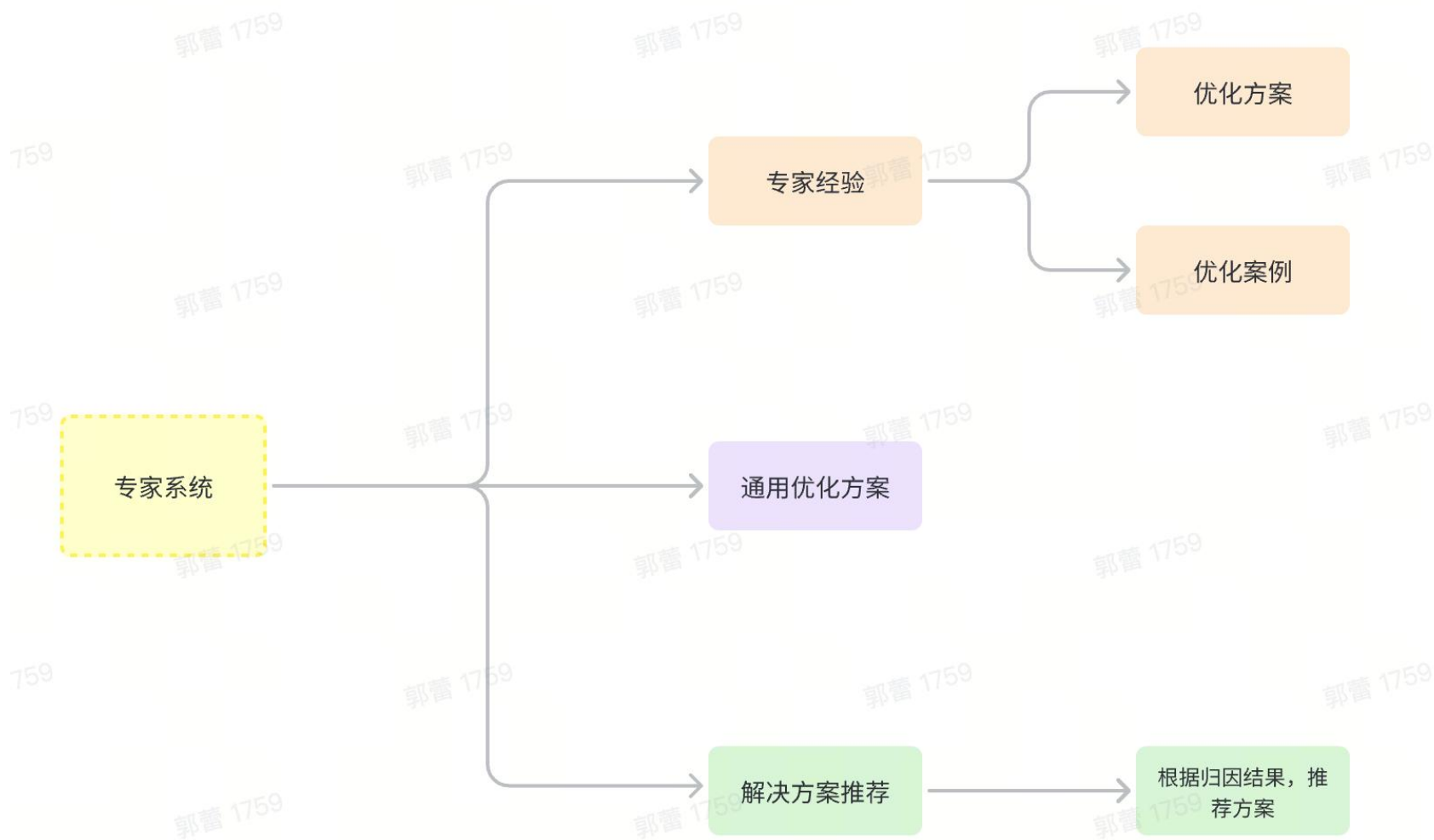
耗时信息

归因结果

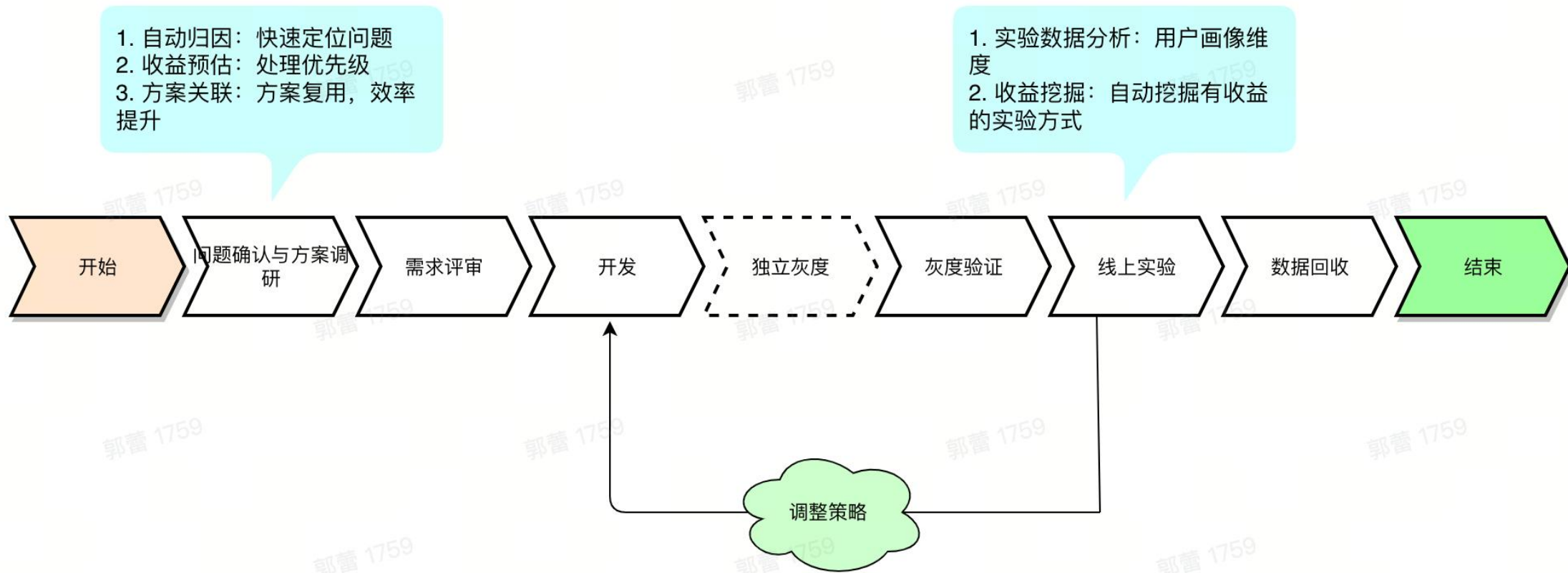
02_ 技术建设-问题消费-消费形态



02_ 技术建设-问题消费-专家系统



02_ 技术建设-问题消费-流程跟进



02_ 技术建设-问题消费-实验分析

主要提供以下能力：

- 支持按照用户画像分析实验，目前支持20+大流量画像，
- 收益自动挖掘，支持对大盘无收益，但特定用户群体有收益的实验进行挖掘

核心指标（全局）					显著性力
实验分组	近30天app活跃天数 ◆	人均app停留时长 ◆	播放时长 ◆	人均播放量(进组人群) ◆	人均播放完毕数量 ◆
✓ 机型打分(10)					
v2(实验组) 影响用户:  影响用户比例: 	0.05005585% 	0.20650777% 	0.22181939% 	0.05412218% 	0.21068138% 
v3(实验组) 影响用户:  5 影响用户比例: 	正向显著 0.08112370% 	0.20843689%  3	0.23742154% 	0.22052122% 	0.28940101% 

03

阶段性成果

03_阶段性成果



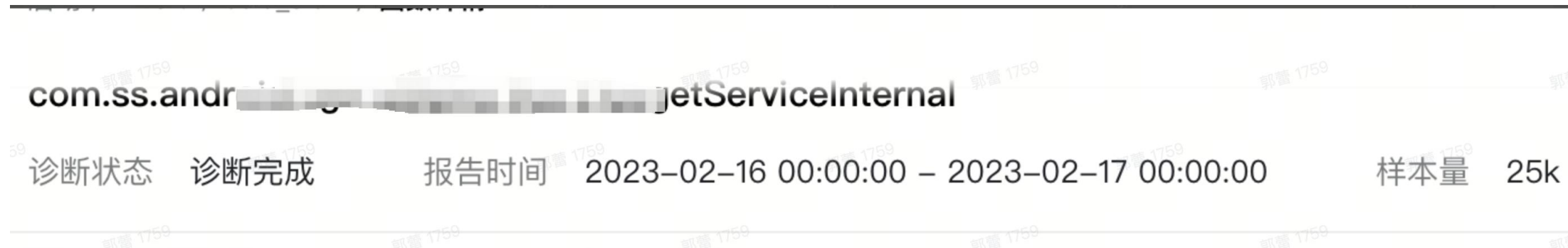
平台效果



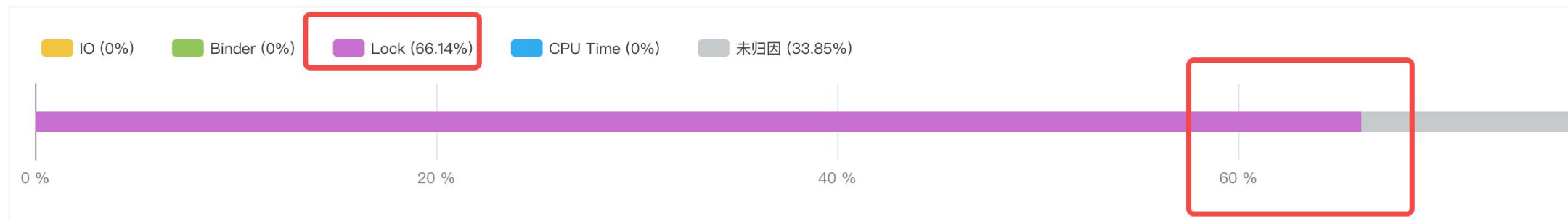
合作收益

03_阶段性成果-平台案例

案例2: 业务代码导致的锁耗时问题。



关键函数归因



03_阶段性成果-平台案例

案例3: 后台任务耗时导致启动慢。

▼ tp-default-21

task Task

CPU time

511ms

510ms

Wall time

1471ms

1464ms

task Task

1ms

7ms

IO

```
try {  
    ZipFile zipFile = new ZipFile(hostApkFile);  
    Set<String> hostAbiSet = getAbiSet(zipFile);  
    zipFile.close();
```

```
    if (contains32Bits(hostAbiSet)) {  
        bits |= ABI_BITS_32;  
    }
```

```
    if (contains64Bits(hostAbiSet)) {  
        bits |= ABI_BITS_64;  
    }
```

03_阶段性成果-平台案例

案例4: 锁问题导致的耗时

▼ com.xx.sdk.core.xx.getXXSync()

▼ com.x.xxx.xxx.xx.sdk.storage.dao.xx.getXX(java.lang.String,java.lang.String)

▼ com.xx..DbManager.rawQuery(java.lang.String,java.lang.String[])

慢查询吗?

归因结果: 锁耗时问题

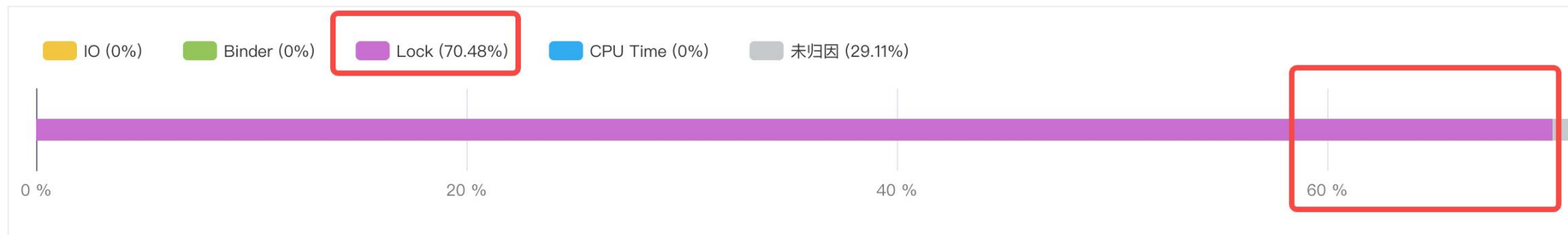
com...:rawQuery

诊断状态 诊断完成

报告时间 2023-03-14 23:08:04 – 2023-03-16 03:08:04

样本量 1k

关键函数归因



03_阶段性成果-合作收益

通过平台发现多例问题， 经过优化之后， 上线多项实验策略。对日活、使用时长等核心指标都显著的提升。



04

总结&展望

04_总结&展望

- 总结
 - 简单的介绍了下启动监控的背景
 - 详细的介绍了抖音启动性能监控平台的建设思路
- 展望
 - 加强启动场景的数据建设，拓展和细化更多维度的性能数据，如内存方向
 - 支持更多用户体验场景，如温启动、首刷卡顿等



THANKS.